# hdoc day 42 by trishil chauhan

## Menu-Driven C Program: String Operations

Aim: Write a menu-driven C program using separate user-defined functions to remove vowels, identify the first repeating lowercase alphabet, and check whether two strings are anagrams.

## Program (C)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000

void readString(char str[], size_t size) {
    if (fgets(str, (int)size, stdin) != NULL) {
        str[strcspn(str, "\n")] = '\0';
    }
}

void removeVowels(const char source[], char result[]) {
    int j = 0;
    for (int i = 0; source[i] != '\0'; i++) {
        char ch = (char)tolower((unsigned char)source[i]);
        if (ch != 'a' && ch != 'e' && ch != 'i' &&
            ch != 'o' && ch != 'u') {
            result[j++] = source[i];
        }
    }
    result[j] = '\0';
}

char firstRepeatingLowercase(const char str[]) {
    int seen[26] = {0};
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] >= 'a' && str[i] <= 'z') {
            int index = str[i] - 'a';
            if (seen[index]) return str[i];
            seen[index] = 1;
        }
    }
    return '\0';
}

int areAnagrams(const char first[], const char second[]) {
    int count[256] = {0};
    for (int i = 0; first[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)first[i];
        if (!isspace(ch)) count[(unsigned char)tolower(ch)]++;
    }
    for (int i = 0; second[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)second[i];
        if (!isspace(ch)) count[(unsigned char)tolower(ch)]--;
    }
    for (int i = 0; i < 256; i++) {
        if (count[i] != 0) return 0;
    }
    return 1;
}

int main(void) {
    char str1[MAX], str2[MAX], result[MAX];
```

```
    int choice;
    do {
        printf("\n--- STRING OPERATIONS MENU ---\n");
        printf("1. Remove all vowels\n");
        printf("2. Find first repeating lowercase alphabet\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\nEnter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input.\n");
            return 1;
        }
        int c;
        while ((c = getchar()) != '\n' && c != EOF) { }

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                readString(str1, sizeof str1);
                removeVowels(str1, result);
                printf("Without vowels: %s\n", result);
                break;
            case 2: {
                printf("Enter a string: ");
                readString(str1, sizeof str1);
                char ch = firstRepeatingLowercase(str1);
                if (ch != '\0')
                    printf("First repeating lowercase alphabet: %c\n", ch);
                else
                    printf("No repeating lowercase alphabet found.\n");
                break;
            }
            case 3:
                printf("Enter first string: ");
                readString(str1, sizeof str1);
                printf("Enter second string: ");
                readString(str2, sizeof str2);
                if (areAnagrams(str1, str2))
                    printf("The strings are anagrams.\n");
                else
                    printf("The strings are not anagrams.\n");
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice. Try again.\n");
        }
    } while (choice != 4);
    return 0;
}
```

## Sample Output

```
--- STRING OPERATIONS MENU ---
1. Remove all vowels
2. Find first repeating lowercase alphabet
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 1
Enter a string: Education
Without vowels: dctn

Enter your choice: 2
Enter a string: programming
First repeating lowercase alphabet: r
```

```
Enter your choice: 3
Enter first string: Listen
Enter second string: Silent
The strings are anagrams.

Enter your choice: 4
Exiting program.
```

## Notes

The repeating-letter operation considers only lowercase a-z and reports the character at its second occurrence. The anagram check ignores letter case and whitespace, but counts punctuation and other characters.